Computing Systems and Programming (4): iocontrol_serialin

Dr. Clifford Choy

School of Design

16 Oct 2026

iocontrol_serialin

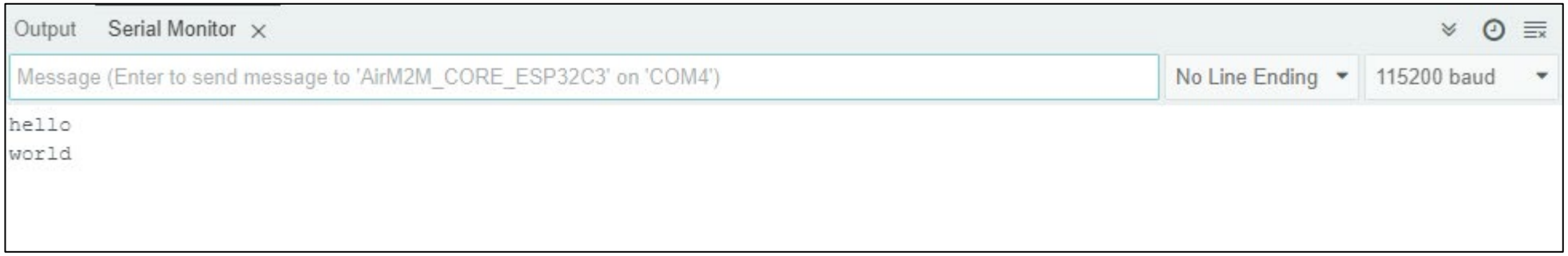
iocontrol_serialin

- `Serial.available()` for checking whether there is anything from Serial Monitor (on PC)
- Read inputs from Serial Monitor using `Serial.readStringUntil()`

```
7   DeclareHandler(hdlr,_ctrl,IOControl,{
8     // if (_ctrl.isSchedulerTimeEvent())
9     //     Serial..println("IOControl - SchedulerTimerEvent");
10    if (Serial.available() > 0) {
11      String input = Serial.readStringUntil('\n');
12      Serial.println(input);
13    }
14  });
```

iocontrol_serialin

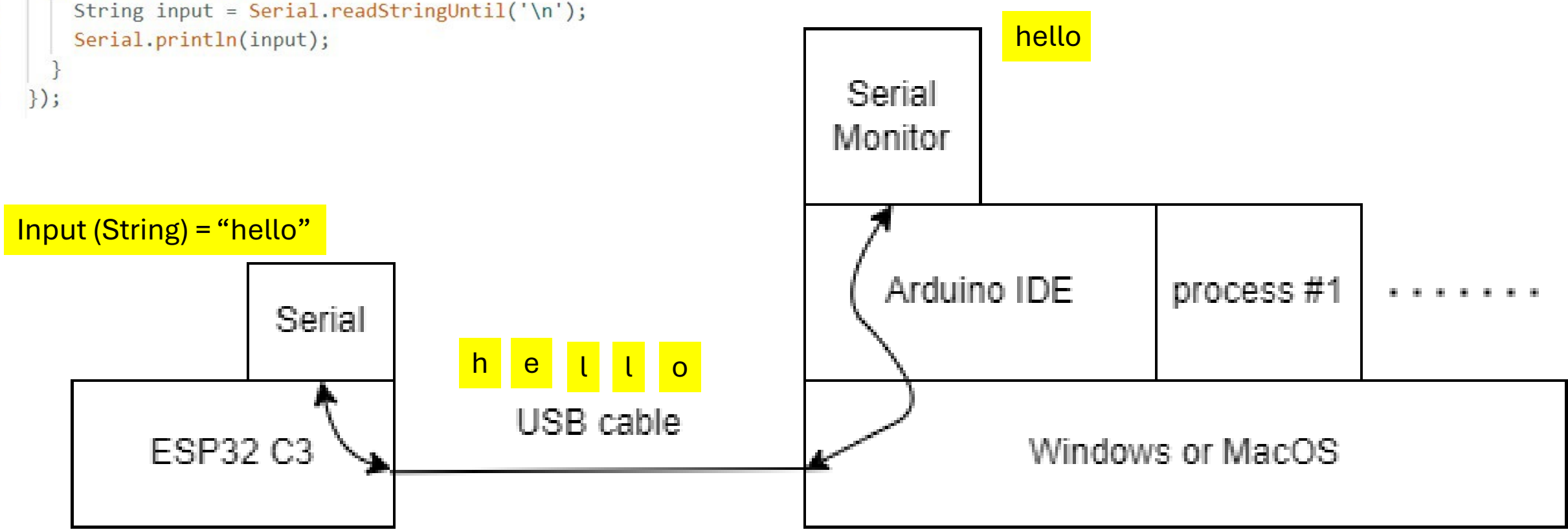




Serial Communication

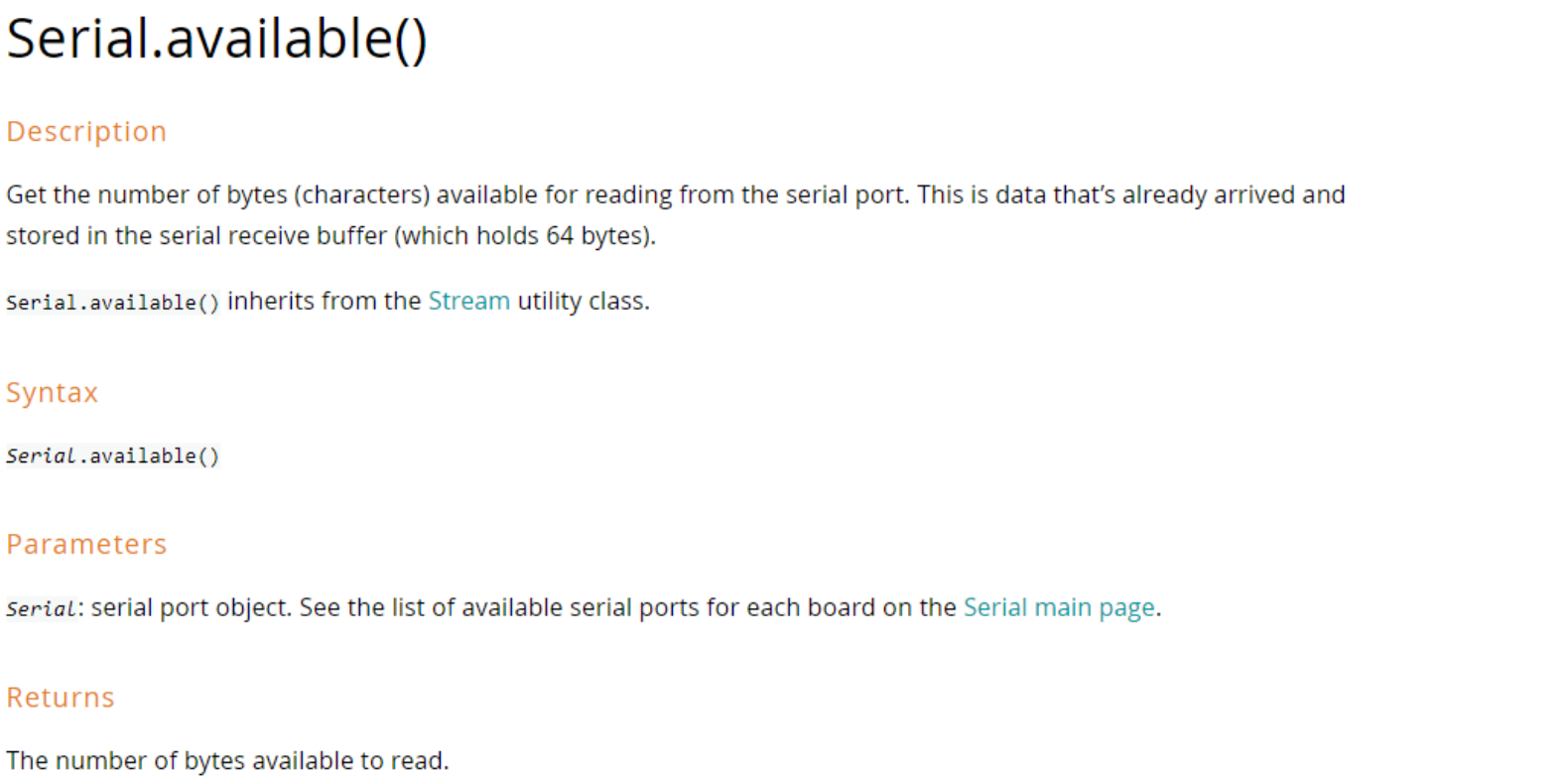
```
7   DeclareHandler(hdlr,_ctrl,IOControl,{
8     // if (_ctrl.isSchedulerTimeEvent())
9     //     Serial..println("IOControl - SchedulerTimerEvent");
10    if (Serial.available() > 0) {
11      String input = Serial.readStringUntil('\n');
12      Serial.println(input);
13    }
14  });
```

- `Serial.available()`
  - how many characters sent from **Serial Monitor**
- `Serial.readStringUntil()`
  - actually read those characters until a `'\n'` character is encountered
  - Not include `'\n'`



Serial.available()

- https://docs.arduino.cc/language-reference/en/functions/communication/serial/available/



Serial.available()

Description

Get the number of bytes (characters) available for reading from the serial port. This is data that's already arrived and stored in the serial receive buffer (which holds 64 bytes).

`Serial.available()` inherits from the Stream utility class.

Syntax

`Serial.available()`

Parameters

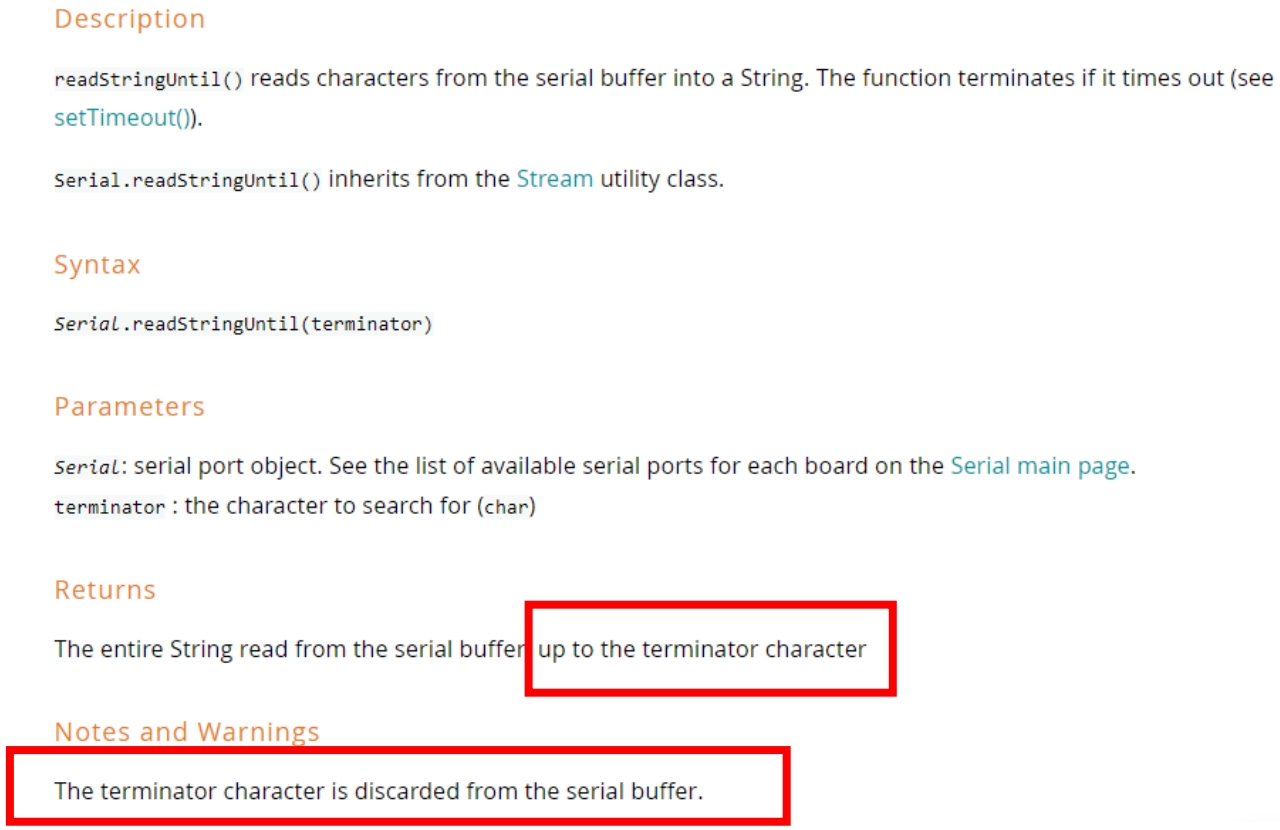
`Serial`: serial port object. See the list of available serial ports for each board on the Serial main page.

Returns

The number of bytes available to read.

Serial.readStringUntil()

- https://docs.arduino.cc/language-reference/en/functions/communication/serial/readStringUntil/



Description

`readStringUntil()` reads characters from the serial buffer into a String. The function terminates if it times out (see setTimeout()).

`Serial.readStringUntil()` inherits from the Stream utility class.

Syntax

`Serial.readStringUntil(terminator)`

Parameters

`Serial`: serial port object. See the list of available serial ports for each board on the Serial main page.
`terminator` : the character to search for (`char`)

Returns

The entire String read from the serial buffer up to the terminator character

Notes and Warnings

The terminator character is discarded from the serial buffer.

Comparison Operators (1)

- Expression with **comparison operators** resulted in **Boolean value** (that is, either `true` or `false`), which can be used in **conditional statement** (e.g. `if ... else`)

Comparison Operators

!= (not equal to)
< (less than)
<= (less than or equal to)
== (equal to)
> (greater than)
>= (greater than or equal to)

https://www.arduino.cc/reference/en/#structure

```
int a=10, b=20, c=30;
float d=10.1, e=10.0, f=20.0;
bool v;

v = a == b;  // = false
v = a > b;   // = false
v = a < c;   // = true
v = d >= e;  // = true
v = e != a;  // = false
```

An expression with comparison operator ==, which resulted in `true` or `false`
Note that == means “is equal to”, NOT an assignment operator

Comparison Operators (2)

```
7   DeclareHandler(hdlr,_ctrl,IOControl,{
8     // if (_ctrl.isSchedulerTimeEvent())
9     //     Serial..println("IOControl - SchedulerTimerEvent");
10    if (Serial.available() > 0) {
11      String input = Serial.readStringUntil('\n');
12      Serial.println(input);
13    }
14  });
```

An expression determining whether `Serial.available()` is **larger than** `0` or not

Boolean Operators (1)

- For use with expressions which resulted in Boolean values

Boolean Operators

! (logical not)
&& (logical and)
|| (logical or)

https://www.arduino.cc/reference/en/#structure

x	!x
true	false
False	true

not x

x	y	x && y
true	true	true
false	true	false
true	false	false
false	false	false

x AND y
True if both x and y are true

x	y	x || y
true	true	true
false	true	true
true	false	true
false	false	false

x OR y
True if either x or y is true

- Note:
  - x, y can be a variable of type “bool”, or
  - x, y can be an expression evaluated to a value of type “bool”

Boolean Operators (2)

- Examples:

```
int a;
bool v1,v2;

v1 = (a>5) && (a<20);  //  true if a is in the range (5,20), but not including 5 or 20
v2 = (a<=5) || (a>=20);  //  true if a is either less than or equal to 5 or larger than or equal to 20
```

a	v1	v2
1	false	true
5	false	true
10	true	false
20	false	true
30	false	true

Conditionals

- For control which statements to be executed

```
int a=10, b=0;
bool c=true;

if ((a>5) && (a<10))
{ Serial.println("hello");
}
Serial.println("world");

if (a!=b)
{ Serial.println("hello");
}else
{ Serial.println("world");
}

if (!b) Serial.println("b is false");

if c Serial.println("c is true");
```

An expression which returns `true` or `false`
Or a number (zero means `false`, and non-zero means `true`)

Possible to have more than one statements inside the block

Possible to have `if` only, without any braces, if only one statement

Possible to have `if` and `else`

If only one statement, no need to use `{` and `}`

`b == 0` and hence is interpreted as `false`
`!b` is then `true`

This is error – must have a bracket

if…else if…

- https://www.arduino.cc/reference/en/language/structure/control-structure/else/

Syntax

```
if (condition1) {
  // do Thing A
}
else if (condition2) {
  // do Thing B
}
else {
  // do Thing C
}
```

Example Code

Below is an extract from a code for temperature sensor system

```
if (temperature >= 70) {
  // Danger! Shut down the system.
}
else if (temperature >= 60) { // 60 <= temperature < 70
  // Warning! User attention required.
}
else { // temperature < 60
  // Safe! Continue usual tasks.
}
```

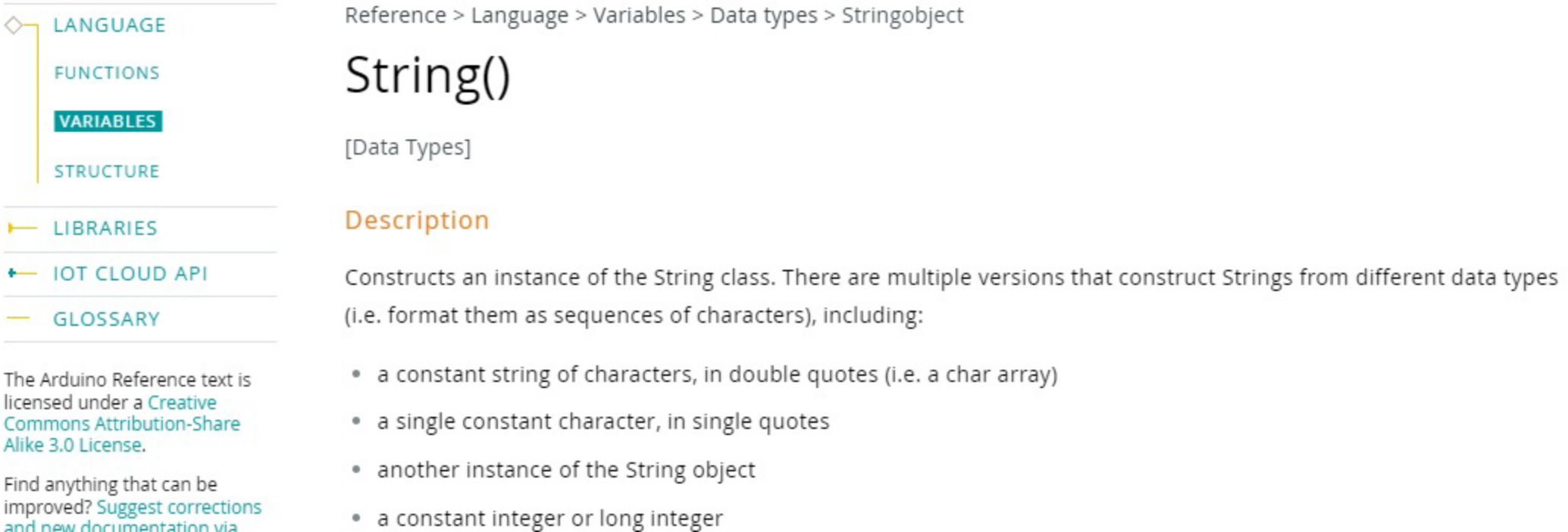
About String (1)

- Variable of type `String` is for storing **sequence of characters**

```
7   DeclareHandler(hdlr,_ctrl,IOControl,{
8     // if (_ctrl.isSchedulerTimeEvent())
9     //     Serial..println("IOControl - SchedulerTimerEvent");
10    if (Serial.available() > 0) {
11      String input = Serial.readStringUntil('\n');
12      Serial.println(input);
13    }
14  });
```

About String (2)

- Refer to https://docs.arduino.cc/language-reference/en/variables/data-types/stringObject/ on various functions/methods and operations for `String`



About String (3)

Functions/methods

charAt()	indexOf()	substring()
compareTo()	lastIndexOf()	toCharArray()
concat()	length()	toDouble()
c_str()	remove()	toInt()
endsWith()	replace()	toFloat()
equals()	reserve()	toLowerCase()
equalsIgnoreCase()	setCharAt()	toUpperCase()
getBytes()	startsWith()	trim()

Operators

- [] (element access)
- + (concatenation)
- += (append)
- == (comparison)
- > (greater than)
- >= (greater than or equal to)
- < (less than)
- <= (less than or equal to)
- != (different from)

Some Frequently Used Functions/Operations

- `concat()` or + or +=
- Comparison operators
  - ==, >, >=, <, <=, !=
  - `equals()`
  - `equalsIgnoreCase()`
- `length()`
- `toInt(),toFloat()`
- `substring()`
- `charAt()`
- `indexOf(),lastIndexOf()`
- `trim()`
- `setCharAt(),replace()`

length() and charAt()

```
7   DeclareHandler(hdlr,_ctrl,IOControl,{
8     // if (_ctrl.isSchedulerTimeEvent())
9     //     Serial..println("IOControl - SchedulerTimerEvent");
10    if (Serial.available() > 0) {
11      String input = Serial.readStringUntil('\n');
12      Serial.println(input);
13
14      int len = input.length();
15      Serial.print("no of characters read =");
16      Serial.println(len);
17
18      int index = input.charAt('a');
19      Serial.print("character \'a\' is at index =");
20      Serial.println(index);
21
22    }
23  });
```

https://www.arduino.cc/reference/en/language/variables/data-types/string/functions/length/

https://www.arduino.cc/reference/en/language/variables/data-types/string/functions/charat/

Compile Example4_1.zip and observe results from Serial Monitor

Task 3

- From https://docs.arduino.cc/language-reference/en/variables/data-types/stringObject/Functions/toInt/ , find the documentation on `toInt()` on what this function is about
- Add appropriate codes to `iocontrol_serialin` so that this function is used to convert the input message into an **integer number**. The integer number is then added by 15, and the resulting number is then printed out to the Serial Monitor.
- Compile and upload your program to ESP32C3
- Under the heading “Task 3” in your submission page, upload the following:
  - Screen-capture the output from Serial Monitor
  - Compressed file of your sketch (in zip format)

Task 4

- Combine `iocontrol_serialin` and `iocontrol_serialout_fsm1` to create a program which performs the following:
  - When the chip starts up, nothing will be shown in Serial Monitor
  - If user enters a positive number (that is, >=1 and must be an integer),
    - The program will print out the sequence of “Hello world !!!” messages as in `iocontrol_serialout_fsm1` for the number of times as specified by the input number
  - Else,
    - Print out a message informing the user that a positive number must be entered
- **Please refer to the example `iocontrol_serialout_fsm2` on how to repeat the sequence of printing messages for a number of times**
- Compile and upload your program to ESP32C3

- Under the heading “Task 4” in your submission page, upload the following:
  - Video recording of the output from Serial Monitor demonstrating the above logic
  - Compressed file of your sketch (in zip format)

Summary

- `Serial.available()`, `Serial.readStringUntil()`
- Comparison operators
- `String`, and its functions/methods and operators